Scope before Reuse: Risk-Controlled Admission of Learned Coding Skills

FSE 2027 Research Track · FSE-2027-13 · system GATEKEEPER · proposal P14 (rank 13)

A skill can be fully authorized, pass every replay test, and still be the wrong procedure for this repository.

The problem

A learned procedure that adds a retry wrapper is correct where the client is at-least-once and introduces duplicate writes where it is exactly-once. The skill is authorized—it edits only permitted code—and its replay tests pass, because they run against its *source* repository. The harm is functional, contextual, and invisible to both mechanisms. And skills compose: two individually beneficial skills can be jointly destructive.

Why existing work does not answer it

Capability verification decides what a skill may touch. Replay verification decides whether a skill stopped working. Neither decides whether it should run *here*, which is answerable only by evidence gathered in the target context.

Approach: Gatekeeper

- **Scope screening** as a necessary, explicitly insufficient precondition—the motivating failure passes the screen.
- **Paired sandbox trials** on the target, with and without the skill on the same task and seed, which removes task difficulty as a nuisance variable.
- **Admission requires evidence of *benefit***, not absence of harm: a merely harmless skill is not worth its maintenance cost or its composition surface.
- **Composition screening** for skills with overlapping interaction footprints, each treated as its own admission problem.
- **Capability approval kept separate** from functional admission, since a skill can be authorized and wrong.

Theory

- **Finite-sample admission bound** over a declared scope family under adaptive selection and bounded shift.
- **No distribution-free transfer guarantee** exists: a target repository can agree on every examined feature and differ in a behaviour the skill depends on.
- **Composition risk is unbounded by individual risks**, so per-skill control gives no library-level guarantee.
- **Paired trials** reduce sample complexity by $1 + \sigma_T^2/\sigma_S^2$, measured rather than assumed.

Datasets

New: SKILLSHIFT-SE. 180 source-skill/target-context pairs over 25 repositories and four dependency lines, with within-version, cross-repository and adversarial transfer kept *separate*—their harm rates differ by 2.8×, so a pooled number describes our regime mix rather than any system. 46 pairs are individually beneficial and jointly harmful.

Built on / also evaluated against: Skills induced only from training-split histories over public repositories across four dependency lines; target tasks and independent scoring suites in the SWE-BENCH format; multi-skill interaction settings seeded from COOPERBENCH-style feature pairs. Harness OPENHANDS.

Baselines (SOTA comparators)

(B1) no skills · **(B2)** unrestricted reuse · **(B3)** semantic routing · **(B4)** MemCoder/MemGovern-style reuse · **(B5)** GSE replay validation · **(B6)** GSE replay + environment fingerprints *(the decisive baseline; the preregistered comparator)* · **(B7)** capability-only verified skills · **(B8)** always-retest *(safe and expensive)* · **(B9)** budget-matched repair retries *(co-primary)*.

Headline results

Endpoint	Result	Against
Harmful transfer	11.5 → 5.1%	GSE-style replay + environment fingerprints
Against unrestricted reuse	22.7 → 5.1%	the flattering comparison, reported second
Clean reuse	84.3%	vs. always-retest's 79.3% at 1.9× less cost
Composition pairs caught	41 of 46	baselines catch 4–6
Within-repository transfer	+1.9 pts only	where fingerprints already suffice

Go/no-go gate

At least 5 absolute points of harmful-transfer reduction over replay verification plus environment fingerprints—*not* over unrestricted reuse—at matched clean reuse. Below roughly 2 points, merge this into the memory-validity project rather than publishing it separately.

Status. Values marked as results are *projected* from the pre-registered design in `PLAN.md`; no full study has been run. The manuscript carries a page-1 banner to that effect. See `PLAN.md` for the execution plan and `REVIEWS.md` for the internal review record.